ECA1403 – EMBEDDED SYSTEMS

CO2 – ANALYTICAL BASED ASSESSMENT

AUTOMATED BOTTLE FILLING LINE

A Modular Embedded Controller with Real-Time Monitoring and SystemC Verification

Submission Details	Information
Course Outcome	CO2
Maximum Marks	100
Submitted by	B.Pawan Kumar
Register Number	192465055
Department	CSE-Cyber Security
Date	06/11/2026

1. INTRODUCTION

An automated bottle filling line transfers empty containers through a controlled production sequence. A conveyor brings each bottle to a filling station, a presence sensor confirms its position, and a metering device measures the liquid delivered through a nozzle. When the preset quantity is reached, the controller closes the valve and releases the bottle to the next station.

The proposed design uses a microcontroller-based embedded controller. Its inputs include bottle detection, liquid flow, tank level and emergency-stop status. Its outputs operate the conveyor, supply pump, filling valve and alarm. A nominal target of 500 mL is used for each bottle.

The purpose of this assignment is to translate the process requirements into a modular Embedded C design, evaluate its timing behaviour, and build a SystemC model that can be used to examine normal and fault scenarios before hardware implementation.

2. PROCESS OVERVIEW AND DESIGN ASSUMPTIONS

1. The conveyor transports one bottle at a time to the filling position.
2. The bottle sensor must be active before the filling sequence is enabled.
3. The tank must be at or above 20% for a new fill cycle; below 10% is treated as a critical condition.
4. The flow measurement is integrated over time to estimate the delivered volume.
5. The valve is normally closed when its electrical command is removed.
6. A bottle is filled to a target of 500 mL. The maximum permitted filling interval is 5.5 seconds.
7. The production objective is 10 bottles per minute, corresponding to a 6-second average cycle.

These values are design assumptions for the assessment. Sensor accuracy, valve closing delay, liquid density, nozzle behaviour and actual conveyor indexing time must be measured during commissioning.

STAGE 1 – ANALYSE THE INDUSTRIAL REQUIREMENTS 1.1 Functional requirements

- Identify a bottle at the filling station and inhibit filling if no bottle is present.
- Stop and hold the conveyor while the bottle is being filled.
- Check the tank level before enabling the pump and valve.
- Open the filling valve only after all permissive conditions are satisfied.
- Calculate delivered volume from valid flow-sensor readings.
- Close the valve when the estimated volume reaches 500 mL.
- Detect low flow, invalid sensor values, excessive filling time and emergency-stop activation.
- Move a successfully filled bottle towards the downstream capping station.
- Indicate warnings and faults through a visual or audible alarm.

1.2 Input and output allocation

Signal	Type	Purpose / expected handling
Bottle present	Digital input	Confirms a container is correctly positioned.
Flow pulse / flow rate	Pulse or analogue input	Used to estimate the delivered liquid volume.
Tank level	Analogue or digital input	Permits filling above 20%; critical below 10%.

Emergency stop	Safety input	Requests immediate shutdown; hardware path is preferred.
Conveyor command	Digital output	Moves bottles between stations.
Valve command	Digital output	Controls liquid delivery; de-energized state should close valve.
Pump command	Digital output	Supplies liquid to the filling nozzle.
Alarm	Digital output	Signals abnormal operating conditions.

1.3 Operating conditions

Condition	Interpretation	Required response
Tank level ≥ 20%	Normal filling permissive	A cycle may start if other checks pass.
Tank level 10–19%	Low-level warning band	Inhibit a new cycle and request refill.
Tank level < 10%	Critical low level	Stop pump, close valve and latch a fault.
Flow rate approximately 100 mL/s	Nominal flow	500 mL is expected in about 5 seconds.
No valid flow response	Possible empty supply, blockage or sensor fault	Stop the cycle after the configured response timeout.
Fill duration > 5.5 s	Filling deadline exceeded	Close valve and raise a fault.
Estimated volume > 505 mL	Potential overfill	Close valve and prevent automatic restart.

1.4 Non-functional requirements

- Deterministic response to emergency and process faults.
- Modular firmware that can be unit-tested and maintained.
- Electrical noise immunity for sensor wiring and suitable protection against liquid splashes.
- Safe output defaults during reset, watchdog expiry or loss of controller power.
- Diagnostic information that helps maintenance staff identify the cause of a stop.

1.5 Timing and throughput calculation

At a nominal flow rate of 100 mL/s, the theoretical time to deliver 500 mL is:

Filling time = Required volume / Flow rate = 500 / 100 = 5.0 seconds.

The target of 10 bottles per minute gives an average production cycle of 60 / 10 = 6.0 seconds per bottle. The nominal time left for indexing and other non-overlapped work is therefore 1.0 second. At the maximum filling duration of 5.5 seconds, only 0.5 second remains. The production target is feasible only if the remaining operations fit this allowance or are performed in parallel.

1.6 Requirement statement

The controller shall position each bottle, verify the tank and safety inputs, deliver 500 mL using flow feedback, and release the bottle after successful filling. It shall close the valve and enter a safe fault condition when a critical level, missing flow, overfill, timeout or emergency-stop condition is detected.

STAGE 2 – MODULAR EMBEDDED C DESIGN

2.1 Proposed firmware layers

The firmware is divided into a hardware abstraction layer, input-processing services, a sequence controller and an alarm/diagnostic service. This arrangement keeps hardware-specific register operations separate from the process logic.

Software unit	Responsibility
HAL / device drivers	Read input pins, capture flow pulses and update output drivers.
Sensor service	Filter inputs, validate ranges and convert pulses into volume.
Sequence manager	Control IDLE, POSITION, CHECK, FILL, COMPLETE and FAULT states.
Safety monitor	Give emergency stop and critical faults highest response priority.
Alarm and diagnostics	Store fault codes and operate warning indicators.
Scheduler	Run periodic tasks without long blocking delays.

2.2 State-machine definition

State	Entry action	Transition condition
IDLE	Valve and pump off; wait for bottle.	Bottle detected → POSITION.
POSITION	Stop conveyor and confirm bottle is stationary.	Position confirmed → CHECK.
CHECK	Check tank, emergency stop and sensor health.	All permissives valid → FILL; otherwise → FAULT or wait.
FILL	Start pump and open valve; integrate flow.	500 mL → COMPLETE; timeout/fault → FAULT.
COMPLETE	Close valve, stop pump and record cycle.	Bottle released → IDLE.
FAULT	Close valve, stop pump and inhibit conveyor.	Manual inspection and reset → IDLE.

2.3 Control algorithm

8. Sample the emergency-stop input first. If active, force the safe outputs immediately.
9. Acquire and validate bottle, tank and flow measurements.
10. Update the estimated volume only when a valid flow sample is available.
11. Evaluate tank limits, flow-response timeout, overfill limit and total fill time.
12. Execute the current state transition and update actuator commands.
13. Record faults and service the watchdog only after the control cycle completes successfully.

2.4 Representative Embedded C implementation

The code below illustrates the state logic and uses placeholder driver functions. A real implementation must replace these functions with microcontroller-specific drivers, calibrated flow conversion, non-blocking timekeeping and verified actuator feedback.

```
#include <stdint.h>
#include <stdbool.h>

#define TARGET_ML       500.0f
#define OVERFILL_ML     505.0f
#define LOW_TANK_PCT     20.0f
```

```
#define CRITICAL_PCT     10.0f
#define MAX_FILL_MS    5500u
#define FLOW_LOSS_MS    200u

typedef enum {
    ST_IDLE, ST_POSITION, ST_CHECK,
    ST_FILL, ST_COMPLETE, ST_FAULT
} fill_state_t;

typedef struct {
    bool bottle_present;
    bool emergency_stop;
    bool flow_valid;
    float flow_ml_s;
    float tank_percent;
} input_t;

/* Hardware-specific functions are implemented in the HAL. */
extern input_t read_inputs(void);
extern uint32_t millis(void);
extern void set_conveyor(bool on);
extern void set_valve(bool open);
extern void set_pump(bool on);
extern void set_alarm(bool on);
extern void watchdog_kick(void);

int main(void)
{
    fill_state_t state = ST_IDLE;
    input_t in;
    float delivered_ml = 0.0f;
    uint32_t fill_start = 0u, last_flow_ms = 0u, last_update = 0u;

    for (;;) {
        uint32_t now = millis();
        in = read_inputs();

        /* Highest-priority shutdown path. */
        if (in.emergency_stop || in.tank_percent < CRITICAL_PCT) {
            state = ST_FAULT;
        }

        switch (state) {
        case ST_IDLE:
            set_valve(false); set_pump(false);
            set_conveyor(false); set_alarm(false);
            if (in.bottle_present) state = ST_POSITION;
            break;

        case ST_POSITION:
            set_valve(false); set_pump(false);
            set_conveyor(false);       /* bottle held at nozzle */
            state = ST_CHECK;          /* add position feedback in hardware */
            break;

        case ST_CHECK:
            if (in.bottle_present && in.tank_percent >= LOW_TANK_PCT) {
                delivered_ml = 0.0f;
                fill_start = now;
                last_flow_ms = now;
                last_update = now;
                state = ST_FILL;
            } else {
                state = ST_FAULT;
            }
            break;

        case ST_FILL:
            set_conveyor(false);
```

```
            set_pump(true); set_valve(true);

            if (in.flow_valid) {
                float dt = (float)(now - last_update) / 1000.0f;
                if (dt > 0.0f && dt < 0.25f)
                    delivered_ml += in.flow_ml_s * dt;
                last_update = now;
                if (in.flow_ml_s > 0.0f) last_flow_ms = now;
            }

            if (delivered_ml >= TARGET_ML) {
                set_valve(false);
                set_pump(false);
                state = ST_COMPLETE;
            } else if (delivered_ml > OVERFILL_ML ||
                       (now - fill_start) > MAX_FILL_MS ||
                       (now - last_flow_ms) > FLOW_LOSS_MS ||
                       !in.bottle_present) {
                state = ST_FAULT;
            }
            break;

        case ST_COMPLETE:
            set_valve(false); set_pump(false);
            set_conveyor(true);       /* release filled bottle */
            state = ST_IDLE;
            break;

        case ST_FAULT:
        default:
            set_valve(false); set_pump(false);
            set_conveyor(false); set_alarm(true);
            /* Fault remains latched until supervised reset. */
            break;
        }

        /* Kick only after the safety and control work has completed. */
        watchdog_kick();
    }
}
```

Implementation note: this compact example is for explaining the control structure, not a certified machine-safety program. In production, the emergency stop should remove hazardous actuator energy through an independent safety circuit. Flow integration should use pulse counts or a calibrated sampling method; task scheduling, sensor plausibility checks, fault reset and output feedback must be completed and tested.

2.5 Benefits of the modular approach

- A sensor driver can be replaced without rewriting the state machine.
- The sequence manager can be tested using simulated input values.
- Fault handling is easier to review because shutdown behaviour is centralized.
- The same application logic can be adapted to another microcontroller with a different HAL.

STAGE 3 – REAL-TIME CONSTRAINT ANALYSIS

3.1 Time-critical activities

The controller must not treat all tasks as equally urgent. Emergency shutdown and actuator de-energization have the highest priority. Sensor acquisition and filling control follow, while display updates and production counters can run at lower priority.

Activity	Suggested interval / trigger	Requirement
Emergency-stop handling	Event-driven	Shutdown response within 50 ms.
Bottle sensor sampling	Every 10 ms	Detect a bottle change promptly.
Flow measurement update	Every 50 ms	Update volume and check flow validity.
Tank-level sampling	Every 100 ms	Prevent dry running and low-level filling.
Filling-control task	Every 50 ms	Update valve and pump decisions.
Flow-loss detection	Event / timeout	Detect missing flow within 200 ms.
Maximum fill timer	Continuously checked	Stop filling by 5.5 seconds.
Watchdog supervision	Less than 500 ms	Recover from a stalled processor.

3.2 Timer configuration example

Assume a 16 MHz clock and a timer prescaler of 16. The timer counter frequency is 16 MHz / 16 = 1 MHz, so one count represents 1 microsecond. A 1 ms scheduler tick therefore requires 1,000 timer counts. The actual reload register value depends on whether the selected timer counts from zero, uses an auto-reload register, or includes an offset.

3.3 Worst-case response considerations

- The 50 ms emergency-stop target should be achieved by a dedicated interrupt or, preferably, a hardware safety path that does not depend on the main loop.
- A 50 ms flow update period leaves limited time for filtering and fault confirmation; the flow-loss timer must be designed so total detection latency stays within 200 ms.
- The fill-time limit must include sensor processing and valve-closing delay. The software command should be issued early enough that the physical flow stops before the maximum permitted interval.
- Long delays, blocking serial output and unbounded loops should not be used in the control task.

3.4 Watchdog and safe-state design

A watchdog can reset the processor if the application stops making progress. However, a reset is not a substitute for safe actuator design. The valve should move to its closed state when its drive is de-energized, and the pump should be disabled by a safe output default. The watchdog should be serviced only after critical checks and state updates have completed.

3.5 Timing conclusion

The stated sampling periods and response deadlines form a preliminary schedule. They must be verified by measuring worst-case execution time, interrupt latency, sensor conversion time and actuator response on the selected hardware. The production-rate calculation also shows that conveyor indexing must be very short or overlap with another operation.

STAGE 4 – SYSTEMC-BASED MODELLING

4.1 Purpose of the model

SystemC provides a C++ modelling environment for describing concurrent hardware and software behaviour. For this application, separate modules can represent the plant sensors, filling controller and actuators. Simulation allows the design team to apply normal and abnormal inputs and inspect the resulting output sequence.

4.2 Model structure

Model component	Modelled behaviour
Bottle source	Generates bottle-present events and bottle indexing intervals.
Tank model	Provides a configurable tank-level value.
Flow model	Produces flow-rate values or a zero-flow fault.
Controller	Implements the state machine and timeout logic.
Valve and pump	Reflect the controller output commands.
Testbench	Applies scenarios and checks expected outputs.

4.3 Example SystemC controller skeleton

```
#include <systemc.h>

SC_MODULE(FillingController) {
    sc_in<bool> clk;
    sc_in<bool> bottle_present;
    sc_in<bool> emergency_stop;
    sc_in<double> flow_ml_s;
    sc_in<double> tank_percent;

    sc_out<bool> conveyor_on;
    sc_out<bool> valve_open;
    sc_out<bool> pump_on;
    sc_out<bool> alarm_on;

    enum State { IDLE, CHECK, FILL, COMPLETE, FAULT };
    State state;
    double volume_ml;
    int elapsed_ticks;

    void step() {
        /* Default outputs are safe unless a state enables them. */
        conveyor_on.write(false);
        valve_open.write(false);
        pump_on.write(false);
        alarm_on.write(false);

        if (emergency_stop.read() || tank_percent.read() < 10.0)
            state = FAULT;

        switch (state) {
        case IDLE:
            if (bottle_present.read()) state = CHECK;
            break;

        case CHECK:
            if (tank_percent.read() >= 20.0) {
                volume_ml = 0.0;
                elapsed_ticks = 0;
                state = FILL;
            } else {
                state = FAULT;
            }
            break;
```

```
        case FILL:
            valve_open.write(true);
            pump_on.write(true);
            volume_ml += flow_ml_s.read() * 0.05; // 50 ms model step
            elapsed_ticks++;
            if (volume_ml >= 500.0) state = COMPLETE;
            else if (elapsed_ticks >= 110 ||
                     !bottle_present.read()) state = FAULT;
            break;

        case COMPLETE:
            conveyor_on.write(true);
            state = IDLE;
            break;

        case FAULT:
        default:
            alarm_on.write(true);
            state = FAULT; // latched until testbench reset
            break;
        }
    }

    SC_CTOR(FillingController)
      : state(IDLE), volume_ml(0.0), elapsed_ticks(0) {
        SC_METHOD(step);
        sensitive << clk.pos();
    }
};
```

This is a modelling skeleton. A complete simulation should add reset handling, flow-valid and flow-loss inputs, explicit valve-closing delay, a latched fault-clear mechanism and assertions in the testbench. The controller should also ensure that the valve command is removed immediately when a transition to COMPLETE or FAULT occurs; the model can be refined to calculate next state and outputs in one cycle.

4.4 Simulation scenarios

Scenario	Stimulus	Expected observation
Normal fill	Bottle present, tank 80%, flow 100 mL/s	Volume reaches 500 mL and valve closes.
Bottle absent	Bottle input remains false	Controller remains idle; pump and valve stay off.
Low tank	Tank level 15%	New filling cycle is inhibited.
Critical tank	Tank level 8%	Fault is latched; pump and valve are off.
Flow failure	Flow reading remains zero	Flow-loss timeout causes safe shutdown.
Long fill	Target not reached by 5.5 s	Valve closes and alarm is asserted.
Emergency stop	Emergency input becomes active	Shutdown path is activated within the required response.

4.5 What simulation can and cannot prove

SystemC simulation can demonstrate state transitions, timing assumptions and responses to selected input sequences. It does not prove the accuracy of a physical flow meter, the mechanical closing time of a valve, or compliance with a machinery safety standard. Those properties require hardware testing and an appropriate safety assessment.

STAGE 5 – FAULT HANDLING, VERIFICATION AND VALIDATION

5.1 Fault response plan

Fault	Detection method	Safe response	Recovery
Low tank	Tank level below 20%	Inhibit next fill; issue warning.	Refill tank and recheck level.
Critical tank	Tank level below 10%	Stop pump and close valve.	Inspect supply and acknowledge fault.
No flow	No valid pulses / zero flow for timeout	Close valve, stop pump and latch alarm.	Inspect sensor, supply and nozzle.
Nozzle blockage	Abnormal flow or fill timeout	Stop filling and isolate process.	Clean nozzle and run a test cycle.
Overfill estimate	Volume exceeds 505 mL	Close valve and quarantine bottle.	Check calibration and valve response.
Emergency stop	Safety input active	Remove actuator energy through safety path.	Manual inspection and deliberate reset.

5.2 Verification test cases

Test ID	Test input	Pass condition
VT01	Bottle detected at station	Conveyor stops and filling permissive is evaluated.
VT02	Tank level 80%	Controller permits a normal cycle.
VT03	Flow rate 100 mL/s	Estimated target is reached in approximately 5 s.
VT04	Tank level 15%	New fill is inhibited and warning is shown.
VT05	Tank level 8%	Pump and valve are commanded off; fault is latched.
VT06	Flow sensor stuck at zero	Flow-loss timeout causes shutdown.
VT07	Fill exceeds 5.5 s	Valve closes and alarm is recorded.
VT08	Emergency stop active	Safety shutdown meets the 50 ms target.
VT09	Main control loop stalls	Watchdog resets controller; outputs remain safe.

5.3 Verification method

14. Unit-test each sensor conversion and actuator driver independently.
15. Run state-machine tests with simulated sensor values.
16. Use SystemC to repeat normal, boundary and fault scenarios.
17. Measure task execution time and response latency on the target microcontroller.
18. Perform hardware-in-the-loop tests with the actual valve, pump and sensors.
19. Record measured results and compare them with the stated requirements.

The test cases above define expected results; they are not claimed as completed test measurements.

STAGE 6 – ENGINEERING EVALUATION AND CONCLUSION

6.1 Embedded C and SystemC comparison

Embedded C firmware	SystemC model
Runs on the selected microcontroller.	Runs in a simulation environment.
Uses device drivers to operate real I/O.	Uses ports, signals and abstract modules.
Must satisfy actual processor and peripheral limits.	Explores concurrent behaviour and timing assumptions.
Used for final machine control.	Used for early design review and verification.

6.2 Design improvements

- Use a calibrated flow meter and independently verify delivered volume during commissioning.
- Add position feedback so the controller does not assume that the bottle is stationary merely because the conveyor command is off.
- Use a hardware safety relay or safety controller for emergency shutdown.
- Add valve-position or flow-after-close monitoring to detect a stuck valve.
- Store fault codes and cycle statistics for maintenance and process improvement.
- Tune conveyor indexing and, where practical, overlap bottle transport with non-hazardous preparation tasks.

6.3 Final conclusion

The automated bottle filling line can be implemented as a modular embedded control system. The sequence manager coordinates bottle positioning, tank checks, flow-based filling and bottle release. Real-time analysis identifies the key deadlines for emergency shutdown, sensor updates, flow-loss detection and maximum fill duration. SystemC provides a useful way to examine controller behaviour under normal and faulty input conditions before connecting the firmware to the production equipment.

The calculations indicate that a 500 mL fill at 100 mL/s takes approximately 5 seconds, leaving only 1 second within a 6-second production cycle. Therefore, the throughput target depends on fast or overlapping bottle indexing. Final acceptance requires hardware measurements, calibrated sensors, validated actuator response and independent safety provisions.

REFERENCES

- Course material: ECA14 – Embedded Systems, CO2 assessment brief.
- ISO/IEC 9899, Programming Languages – C (language reference for Embedded C implementation).
- Accellera Systems Initiative, SystemC Language Reference Manual.
- Selected microcontroller datasheet and reference manual (to be specified for the final hardware platform).
- Flow-meter and actuator manufacturer datasheets (to be selected and validated during detailed design).